DATA STRUCTURE – 6

1. Simulate undo and redo operations in a word-processing application using a doubly linked list.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct N{char text[100];struct N *prev,*next;}N;
N *head=NULL,*cur=NULL;
void add(char *s){N*n=malloc(sizeof(N));strcpy(n->text,s);n->prev=cur;n->next=NULL;
 if(cur){N*t=cur->next;while(t){N*x=t->next;free(t);t=x;}cur->next=n;}else head=n;cur=n;}
void undo(){if(cur&&cur->prev)cur=cur->prev;}
void redo(){if(cur&&cur->next)cur=cur->next;}
int main(){add("Hello");add("Hello world");undo();printf("Undo: %s\\n",cur->text);
 redo();printf("Redo: %s\\n",cur->text);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nUndo: Hello
Redo: Hello world
```

2. Given two sorted lists L1 and L2, compute L1 intersection L2 and L1 union L2 using basic list operations.

Program (C)

```
#include <stdio.h>
int main(){int a[]={1,3,5,7},b[]={3,4,5,8},i=0,j=0;
 printf("Intersection: ");
 while(i<4&&j<4){if(a[i]==b[j]){printf("%d ",a[i]);i++;j++;}
 else if(a[i]<b[j])i++;else j++;}
 i=j=0;printf("\\nUnion: ");
 while(i<4||j<4){int x;
 if(j==4||(i<4&&a[i]<b[j]))x=a[i++];
 else if(i==4||b[j]<a[i])x=b[j++];
 else{x=a[i];i++;j++;}
 printf("%d ",x);}return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nIntersection: 3 5
Union: 1 3 4 5 7 8
```

3. Swap two elements by adjusting only pointers (not data) in a doubly linked list.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int d;struct N*p,*n;}N;
void link(N*a,N*b){if(a)a->n=b;if(b)b->p=a;}
void swap(N**h,N*a,N*b){
 if(!a||!b||a==b)return;
 if(a->n==b){N*ap=a->p,*bn=b->n;link(ap,b);link(b,a);link(a,bn);}
 else if(b->n==a)swap(h,b,a);
 else{N*ap=a->p,*an=a->n,*bp=b->p,*bn=b->n;
 link(ap,b);link(b,an);link(bp,a);link(a,bn);}
 if(!a->p)*h=a;if(!b->p)*h=b;
}
void show(N*h){for(;h;h=h->n)printf("%d ",h->d);}
int main(){N x={1},y={2},z={3},w={4};N*h=&x;
 link(&x,&y);link(&y,&z);link(&z,&w);swap(&h,&y,&w);show(h);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\n1 4 3 2
```

4. Implement a self-adjusting list using an array. When an element is accessed, move it to the front without changing the order of the other elements.

Program (C)

```
#include <stdio.h>
int main(){int a[]={10,20,30,40,50},n=5,key=40,pos=-1;
 for(int i=0;i<n;i++){if(a[i]==key){pos=i;break;}}
 if(pos>=0){int v=a[pos];for(int i=pos;i>0;i--)a[i]=a[i-1];a[0]=v;}
 printf("List: ");for(int i=0;i<n;i++)printf("%d ",a[i]);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nList: 40 10 20 30 50
```

5. Write a C program to reverse a given doubly linked list.

Program (C)

```
#include <stdio.h>
typedef struct N{int d;struct N*p,*n;}N;
int main(){N a={1},b={2},c={3};a.n=&b;b.p=&a;b.n=&c;c.p=&b;
 N*t=&a,*h=NULL;while(t){N*x=t->n;t->n=t->p;t->p=x;h=t;t=x;}
 for(t=h;t;t=t->n)printf("%d ",t->d);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\n3 2 1
```

6. Write a C program to delete the middle element of a doubly linked list.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int d;struct N*p,*n;}N;
int main(){N*a=malloc(sizeof*N),*b=malloc(sizeof*N),*c=malloc(sizeof*N);
 a->d=1;a->p=NULL;a->n=b;b->d=2;b->p=a;b->n=c;c->d=3;c->p=b;c->n=NULL;
 N*slow=a,*fast=a;while(fast&&fast->n){slow=slow->n;fast=fast->n->n;}
 if(slow->p)slow->p->n=slow->n;if(slow->n)slow->n->p=slow->p;
 N*t=a;while(t){printf("%d ",t->d);t=t->n;}free(slow);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\n2 is deleted
1 3
```

7. Remove duplicate elements from (i) a sorted DLL and (ii) an unsorted DLL.

Program (C)

```
/* Sorted list: remove adjacent duplicates. */
#include <stdio.h>
typedef struct N{int d;struct N*p,*n;}N;
void unique(N*h){while(h&&h->n){if(h->d==h->n->d){N*t=h->n;h->n=t->n;if(t->n)t->n->p=h;}else h=h->n;}}
int main(){N a={1,0},b={1,&a},c={2,&b},d={2,&c},e={3,&d};
 a.n=&b;b.n=&c;c.n=&d;d.n=&e;e.n=0;unique(&a);
 for(N*t=&a;t;t=t->n)printf("%d ",t->d);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nSorted DLL output: 1 2 3
For an unsorted DLL, compare each node with later nodes and unlink every matching node.
```

8. Calculate the frequency count of a given integer in a circular doubly linked list.

Program (C)

```
#include <stdio.h>
typedef struct N{int d;struct N*p,*n;}N;
int main(){N a={2},b={5},c={2},d={7};a.n=&b;b.n=&c;c.n=&d;d.n=&a;
 a.p=&d;b.p=&a;c.p=&b;d.p=&c;int key=2,count=0;N*t=&a;
 do{if(t->d==key)count++;t=t->n;}while(t!=&a);
 printf("Frequency of %d = %d",key,count);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nFrequency of 2 = 2
```

9. Search for a particular element in a binary search tree.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int d;struct N*l,*r;}N;
N*ins(N*t,int x){if(!t){t=malloc(sizeof*N);t->d=x;t->l=t->r=NULL;}
 else if(x<t->d)t->l=ins(t->l,x);else t->r=ins(t->r,x);return t;}
int find(N*t,int x){if(!t)return 0;if(t->d==x)return 1;return x<t->d?find(t->l,x):find(t->r,x);}
int main(){int v[]={40,20,60,10,30,50,70};N*t=NULL;for(int i=0;i<7;i++)t=ins(t,v[i]);
 int key=50;printf(find(t,key)?"Found":"Not found");return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nFound
```

10. Calculate the height (maximum depth) of a binary tree and try it.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int d;struct N*l,*r;}N;
N*node(int x){N*t=malloc(sizeof*N);t->d=x;t->l=t->r=NULL;return t;}
int height(N*t){if(!t)return 0;int a=height(t->l),b=height(t->r);return 1+(a>b?a:b);}
int main(){N*r=node(1);r->l=node(2);r->r=node(3);r->l->l=node(4);
 printf("Height = %d levels",height(r));return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nHeight = 3 levels
```

11. Convert a binary tree to a doubly linked list, keeping inorder sequence.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int d;struct N*l,*r;}N;
N*new(int x){N*t=malloc(sizeof*N);t->d=x;t->l=t->r=NULL;return t;}
void convert(N*t,N**prev,N**head){if(!t)return;convert(t->l,prev,head);
 if(!*prev)*head=t;else{(*prev)->r=t;t->l=*prev;}*prev=t;convert(t->r,prev,head);}
int main(){N*r=new(2);r->l=new(1);r->r=new(3);N*prev=NULL,*head=NULL;
 convert(r,&prev,&head);for(N*t=head;t;t=t->r)printf("%d ",t->d);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\n1 2 3
```

12. Given a binary tree, connect nodes at the same level using a next-right pointer.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int d;struct N*l,*r,*next;}N;
N*new(int x){N*t=malloc(sizeof*N);t->d=x;t->l=t->r=t->next=NULL;return t;}
void connect(N*r){if(!r)return;N*q[100];int f=0,b=0;q[b++]=r;
 while(f<b){int end=b;N*prev=NULL;while(f<end){N*t=q[f++];if(prev)prev->next=t;prev=t;
 if(t->l)q[b++]=t->l;if(t->r)q[b++]=t->r;}}}
int main(){N*r=new(1);r->l=new(2);r->r=new(3);r->l->l=new(4);r->l->r=new(5);
 connect(r);N*t=r->l;while(t){printf("%d ",t->d);t=t->next;}return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\n2 3
```

13. Implement a priority queue using a binary search tree and perform insertion, deletion of highest priority, and display.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int p;struct N*l,*r;}N;
N*add(N*t,int p){if(!t){t=malloc(sizeof*N);t->p=p;t->l=t->r=NULL;}
 else if(p<t->p)t->l=add(t->l,p);else t->r=add(t->r,p);return t;}
N*popMax(N*t){if(!t)return NULL;if(!t->r){N*x=t->l;free(t);return x;}t->r=popMax(t->r);return t;}
void desc(N*t){if(t){desc(t->r);printf("%d ",t->p);desc(t->l);}}
int main(){N*t=NULL;t=add(t,2);t=add(t,5);t=add(t,3);printf("Priority: ");desc(t);
 t=popMax(t);printf("\\nAfter removing max: ");desc(t);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nPriority: 5 3 2
After removing max: 3 2
```

14. In a binary search tree, delete a node having zero children, one child, or two children.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int d;struct N*l,*r;}N;
N*add(N*t,int x){if(!t){t=malloc(sizeof*N);t->d=x;t->l=t->r=NULL;}
 else if(x<t->d)t->l=add(t->l,x);else t->r=add(t->r,x);return t;}
N*del(N*t,int x){if(!t)return NULL;if(x<t->d)t->l=del(t->l,x);else if(x>t->d)t->r=del(t->r,x);
 else{if(!t->l||!t->r){N*q=t->l?t->l:t->r;free(t);return q;}
 N*q=t->r;while(q->l)q=q->l;t->d=q->d;t->r=del(t->r,q->d);}return t;}
void in(N*t){if(t){in(t->l);printf("%d ",t->d);in(t->r);}}
int main(){int a[]={50,30,70,20,40,60,80};N*t=NULL;for(int i=0;i<7;i++)t=add(t,a[i]);
 t=del(t,30);in(t);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\n40 50 60 70 80
```

15. Convert a given postfix expression to an expression tree.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
typedef struct N{char c;struct N*l,*r;}N;
N*st[100];int top=-1;
N*node(char c){N*t=malloc(sizeof*N);t->c=c;t->l=t->r=NULL;return t;}
void pre(N*t){if(t){printf("%c ",t->c);pre(t->l);pre(t->r);}}
```

```
int main(){char s[]="ab+c*";for(int i=0;s[i];i++){N*t=node(s[i]);
 if(isalnum(s[i]))st[++top]=t;else{t->r=st[top--];t->l=st[top--];st[++top]=t;}}
 printf("Prefix: ");pre(st[top]);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nPrefix: * + a b c
```

16. Construct a binary search tree using inorder, preorder, and postorder representations.

Program (C)

```
/* Build BST from its insertion order (preorder-style keys), then print traversals. */
#include <stdio.h>
#include <stdlib.h>
typedef struct N{int d;struct N*l,*r;}N;
N*add(N*t,int x){if(!t){t=malloc(sizeof*N);t->d=x;t->l=t->r=NULL;}
 else if(x<t->d)t->l=add(t->l,x);else t->r=add(t->r,x);return t;}
void in(N*t){if(t){in(t->l);printf("%d ",t->d);in(t->r);}}
void pre(N*t){if(t){printf("%d ",t->d);pre(t->l);pre(t->r);}}
void post(N*t){if(t){post(t->l);post(t->r);printf("%d ",t->d);}}
int main(){int a[]={40,20,10,30,60,50,70};N*t=NULL;for(int i=0;i<7;i++)t=add(t,a[i]);
 printf("Inorder: ");in(t);printf("\\nPreorder: ");pre(t);printf("\\nPostorder: ");post(t);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nInorder: 10 20 30 40 50 60 70
Preorder: 40 20 10 30 60 50 70
Postorder: 10 30 20 50 70 60 40
```

17. Implement heap sort for a set of numbers.

Program (C)

```
#include <stdio.h>
void heapify(int a[],int n,int i){int m=i,l=2*i+1,r=2*i+2;
 if(l<n&&a[l]>a[m])m=l;if(r<n&&a[r]>a[m])m=r;
 if(m!=i){int t=a[i];a[i]=a[m];a[m]=t;heapify(a,n,m);}}
int main(){int a[]={50,20,30,5,15,22,32},n=7;
 for(int i=n/2-1;i>=0;i--)heapify(a,n,i);
 for(int i=n-1;i>0;i--){int t=a[0];a[0]=a[i];a[i]=t;heapify(a,i,0);}
 printf("Sorted: ");for(int i=0;i<n;i++)printf("%d ",a[i]);return 0;}
```

Input / Output

```
Input: Values are initialized in the program.\nOutput:\nSorted: 5 15 20 22 30 32 50
```